

# Engineering Problem Solving in Python

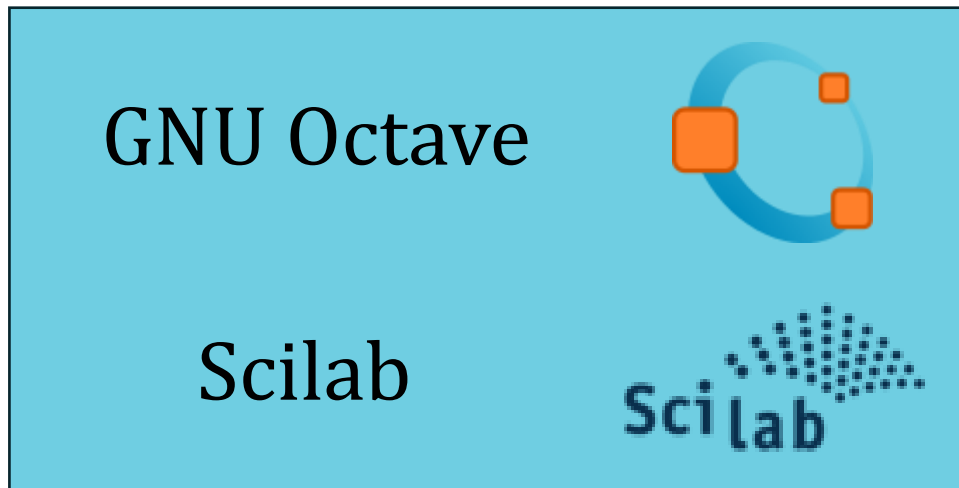
Austin R.J. Downey



# Languages for Scientific Computing



open source “copies”



Wolfram  
Mathematica



R (programming  
language)



Maple™

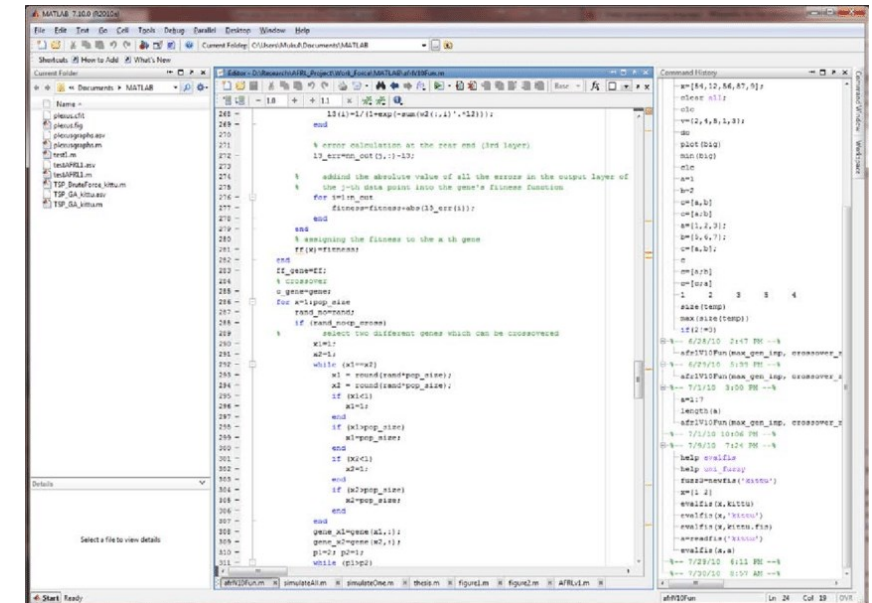
# MATLAB in Engineering

## Pros:

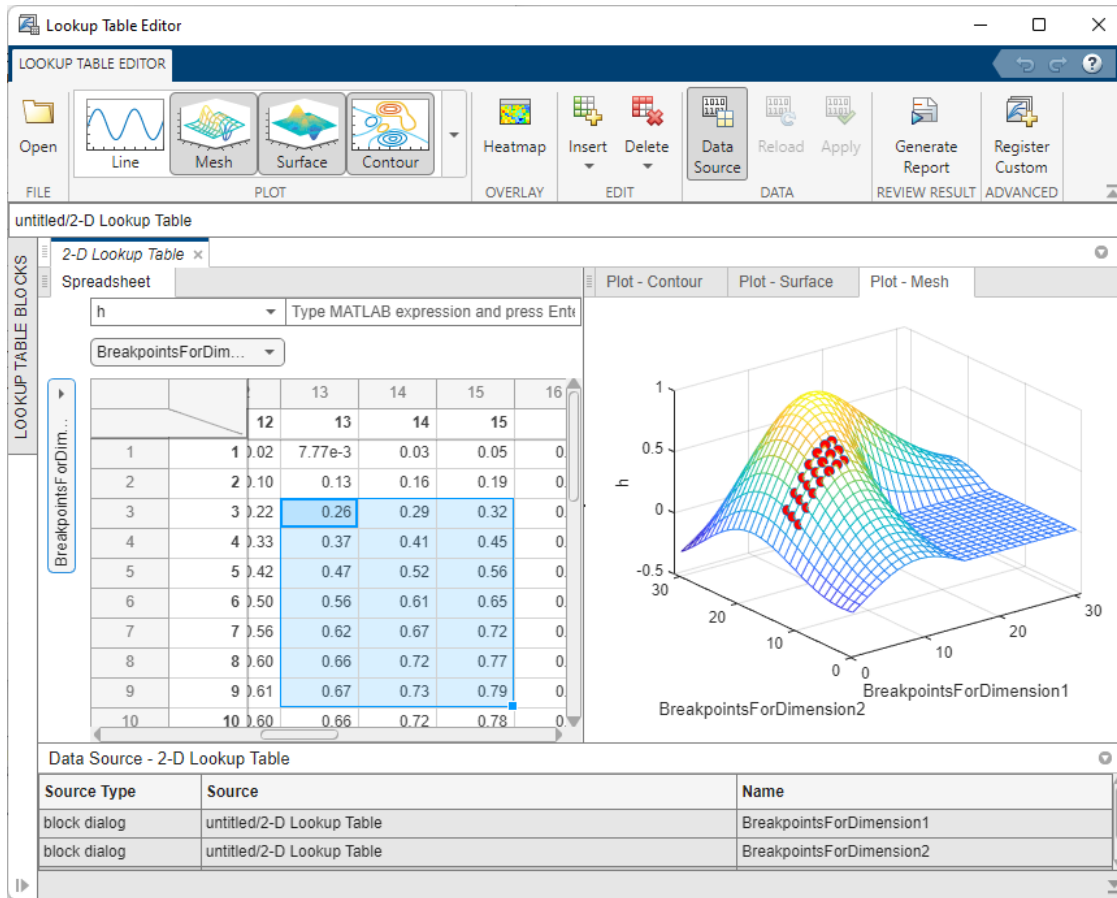
- Works out of the box.
- Code is well managed and packages don't conflict.
- Widely used in engineering and computing.
- Well formulated for linear algebra with simple commands
- Fast implementation of code for complex problem solving
- Not at the bleeding edge of machine learning

## Cons:

- Not free as in speech, not free as in beer
- Each module must be paid for separately
- Not at the bleeding edge of machine learning
- Difficult to keep license updated on multiple machines
- Lets you write poorly structured code.

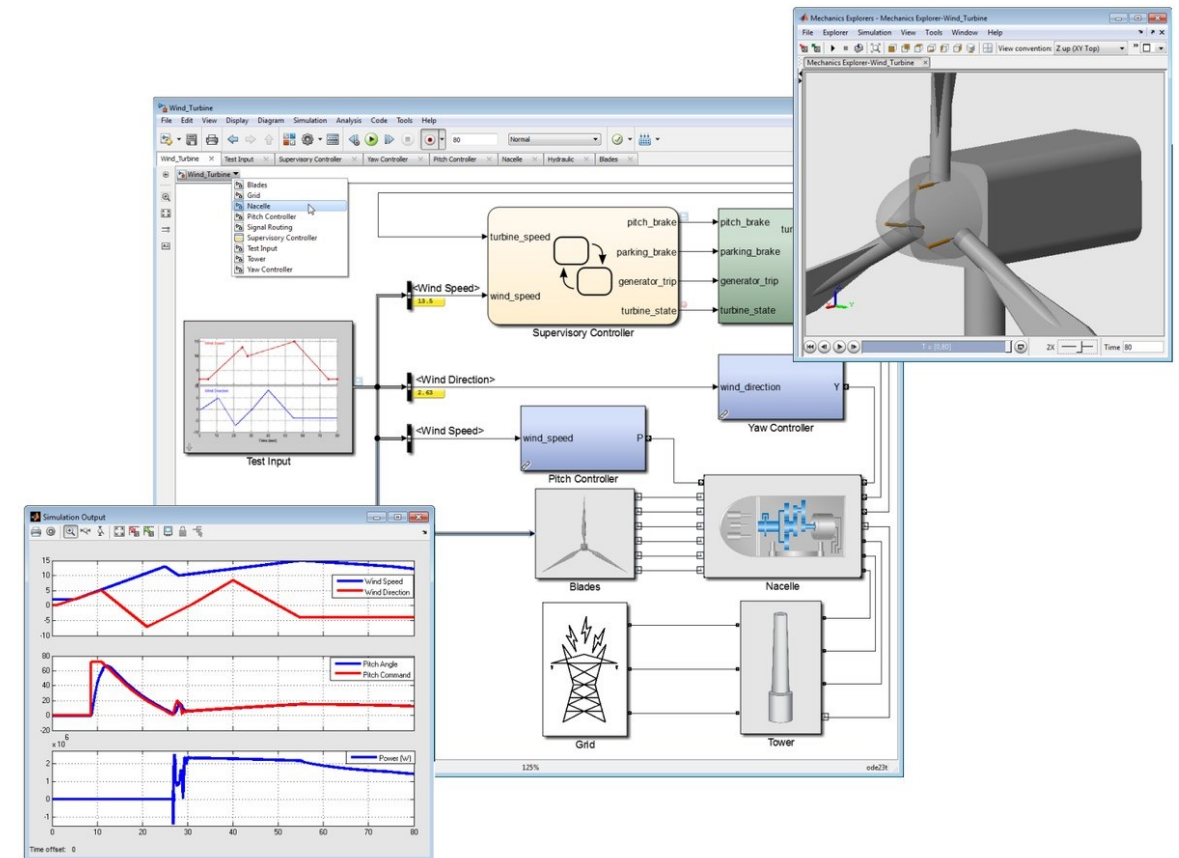


# MATLAB in Engineering



Matlab, via matlab.com

4

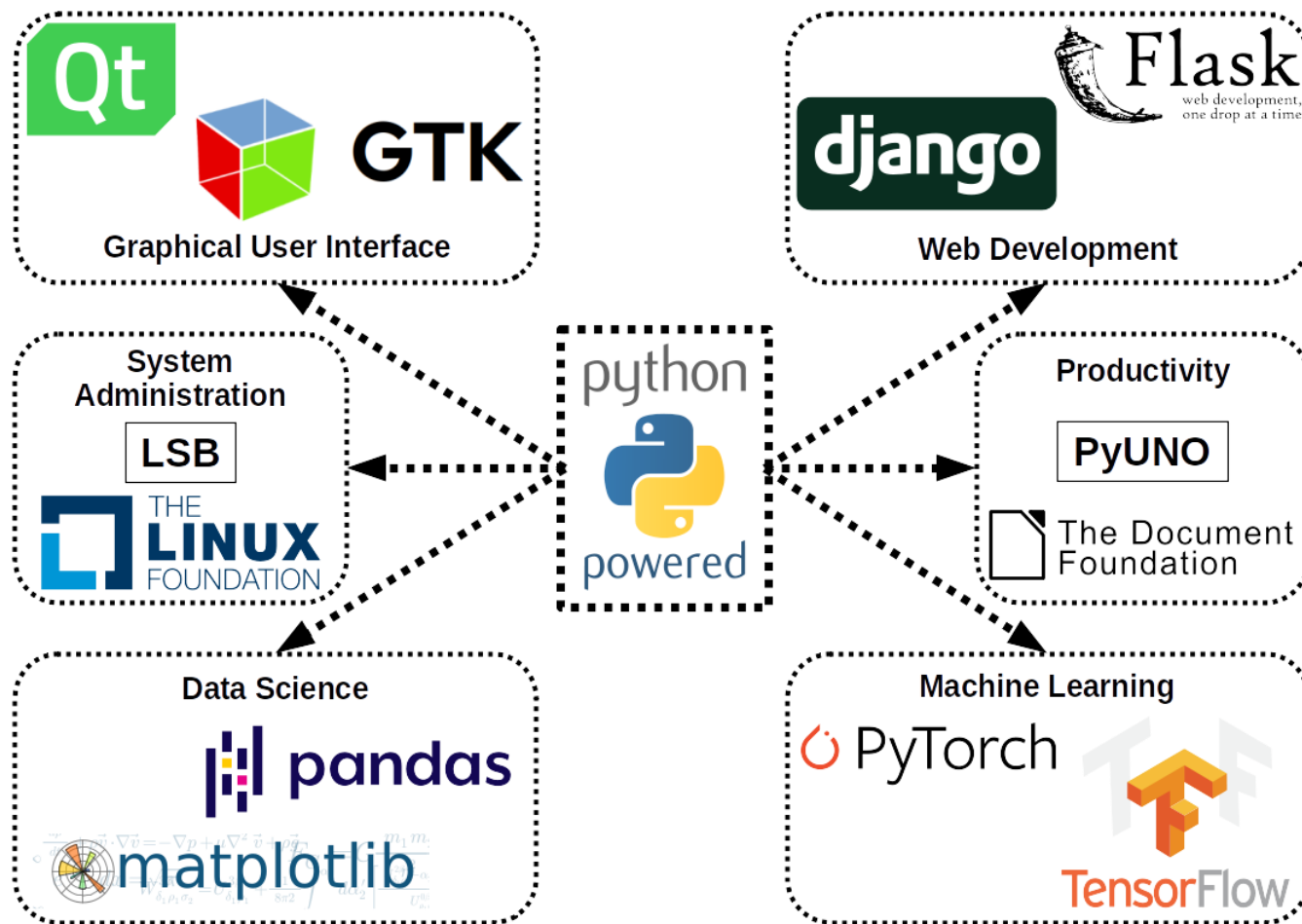


Mcarone1, CC BY-SA 3.0

<<https://creativecommons.org/licenses/by-sa/3.0>>, via Wikimedia Commons

# Python

Python is scripting language that can be used for Scientific Computing.



Guido van Rossum



[https://upload.wikimedia.org/wikipedia/commons/6/66/Guido\\_van\\_Rossum\\_OSCON\\_2006.jpg](https://upload.wikimedia.org/wikipedia/commons/6/66/Guido_van_Rossum_OSCON_2006.jpg)

Named after *Monty Python's Flying Circus*

# Python

Python is an **interpreted** (mostly), **high-level**, **general-purpose** programming language with an **object-oriented** approach.

- **Interpreted language:** is a type of programming language for which its instructions execute directly and freely, without previously compiling a program into machine-language instructions.
- **High-level:** a programming language with strong abstraction from the details of the computer.
- **General-purpose:** is a programming language designed to be used for writing software in the widest variety of application domains (a general-purpose language).
- **Object-oriented:** is a programming paradigm based on the concept of "objects", which can contain data, in the form of fields (often known as attributes or properties), and code, in the form of procedures (often known as methods).



# Python

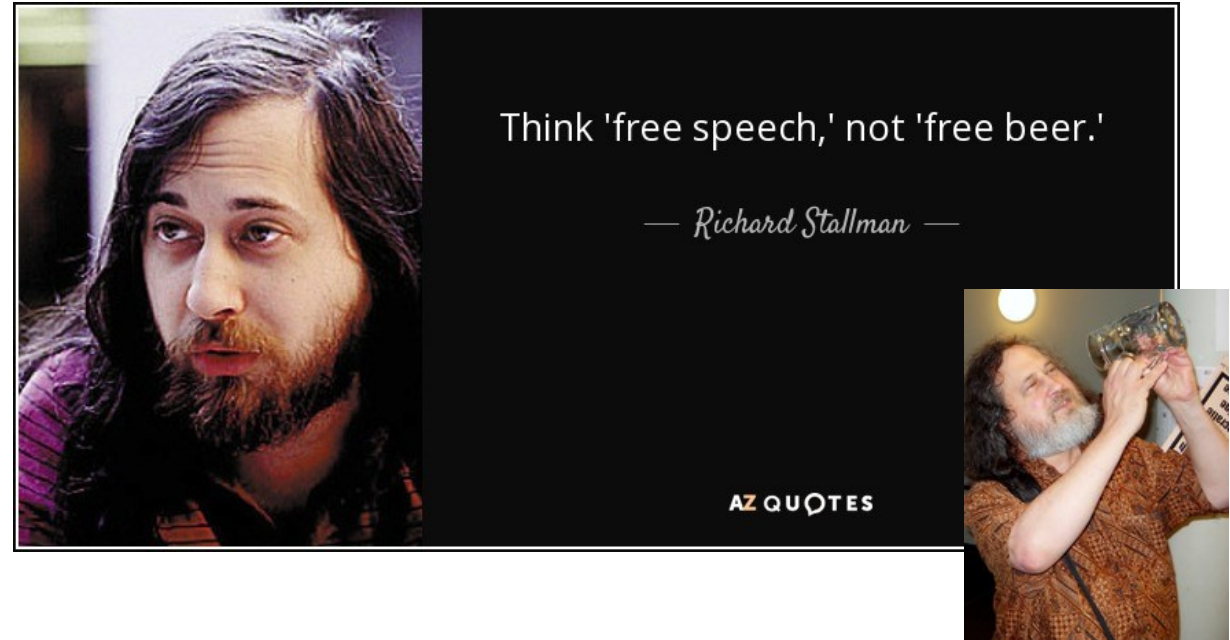
Python's design philosophy emphasizes **code readability** with its notable use of significant **whitespace**. Python is **dynamically typed** and **garbage-collected**.

- **Code readability:** refers to the ease with which a human reader can comprehend the purpose, control flow, and operation of source code. It affects the aspects of quality, including portability, usability and most importantly maintainability.
- **Whitespace:** refers to the usage of whitespace in typing code. In Python, the information about blocks are expressed by their indentation.
- **Dynamically Typed:** refers to languages execute many common programming behaviors at runtime that static programming languages perform during compilation. For us, this means defining variable types.
- **Garbage-Collected:** is a form of automatic memory management. The garbage collector attempts to reclaim garbage, or memory occupied by objects that are no longer in use by the program.

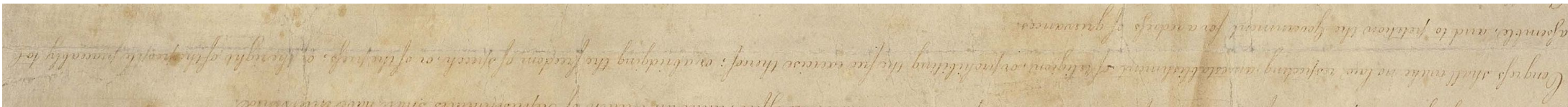


# Python is Free as in beer and Free as in speech

Free beer (never)

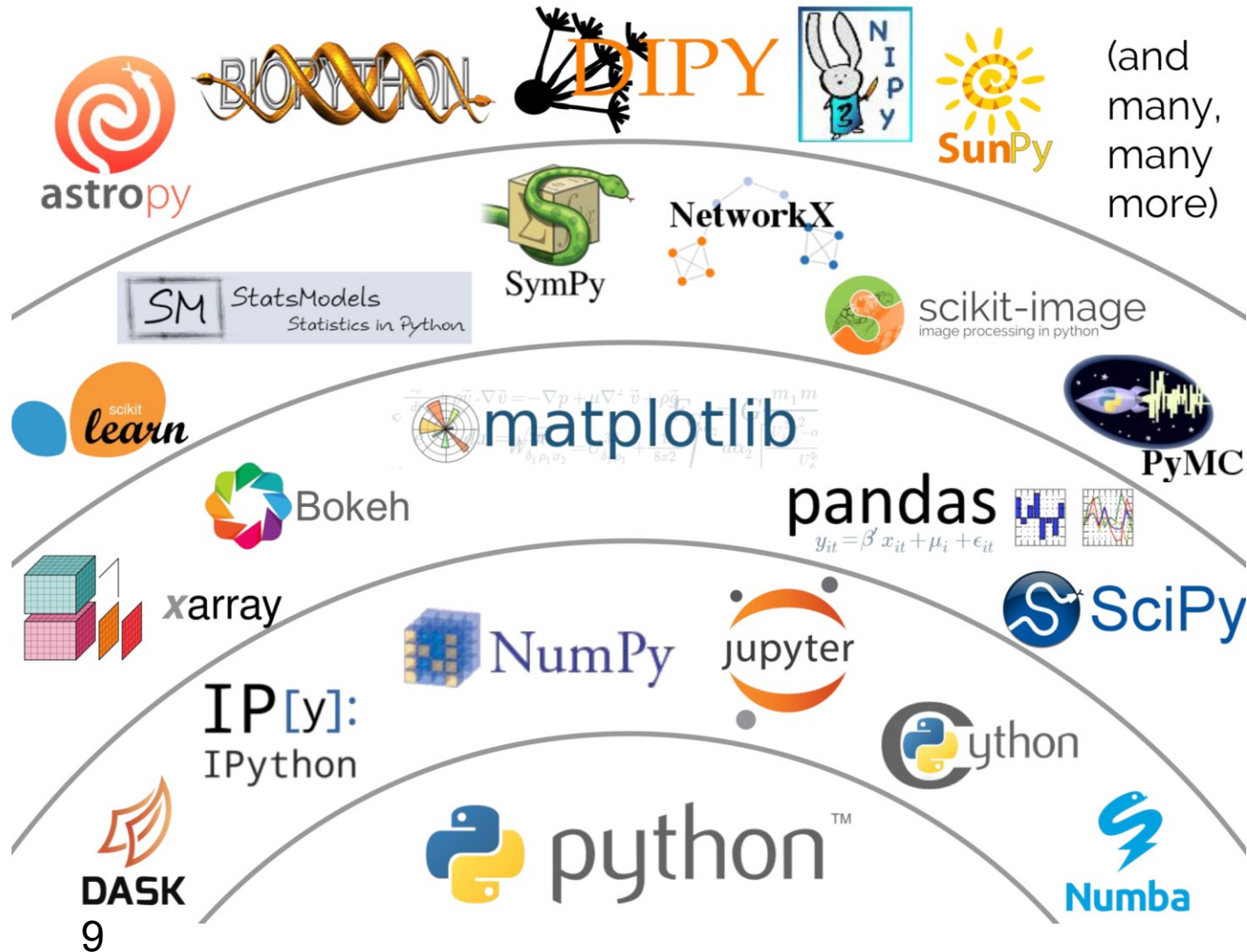


Free speech (First Amendment to the United States Constitution; December 15, 1791)





# Python is a language, we use an ecosystem



In this class we will focus on a limited subset of this ecosystem:

- **IPython:** command shell for interactive computing in Python.
- **Spyder:** cross-platform integrated development environment (IDE) for scientific programming in Python.
- **Numpy:** library adding support for multi-dimensional arrays.
- **SciPy:** library used for scientific computing and technical computing.
- **matplotlib:** API focused on plotting
- **scikit-learn:** library for machine learning.
- **TensorFlow:** symbolic math library, used for neural networks.

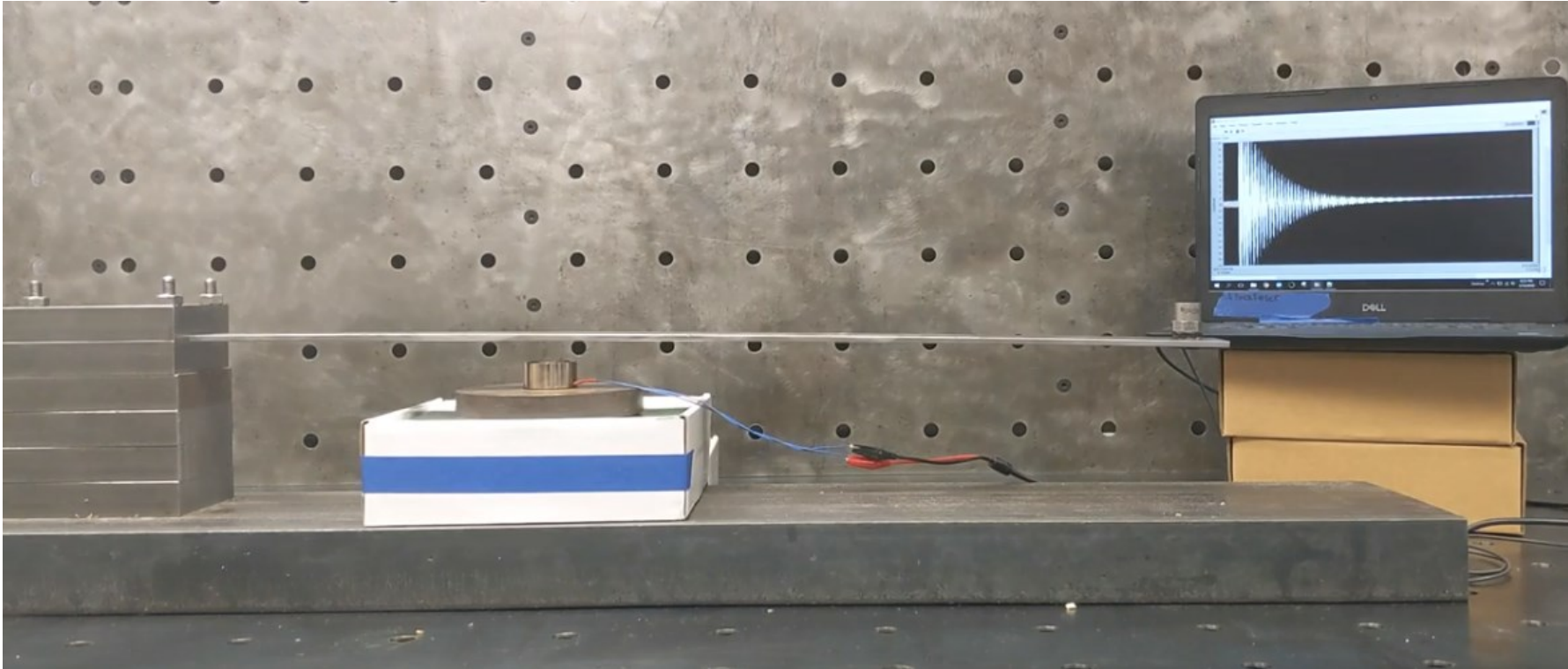
# Python Distributions

A distribution is a group of packages that are assembled and more-or-less work well together. Some of these are free (beer/speech) while for others there are required service packs for their use. In this class, we will use Anaconda from Anaconda, Inc. Anaconda is a free and open-source distribution for scientific computing.



**ActivePython®**

# Example #1 – Plot beam vibration data in Python

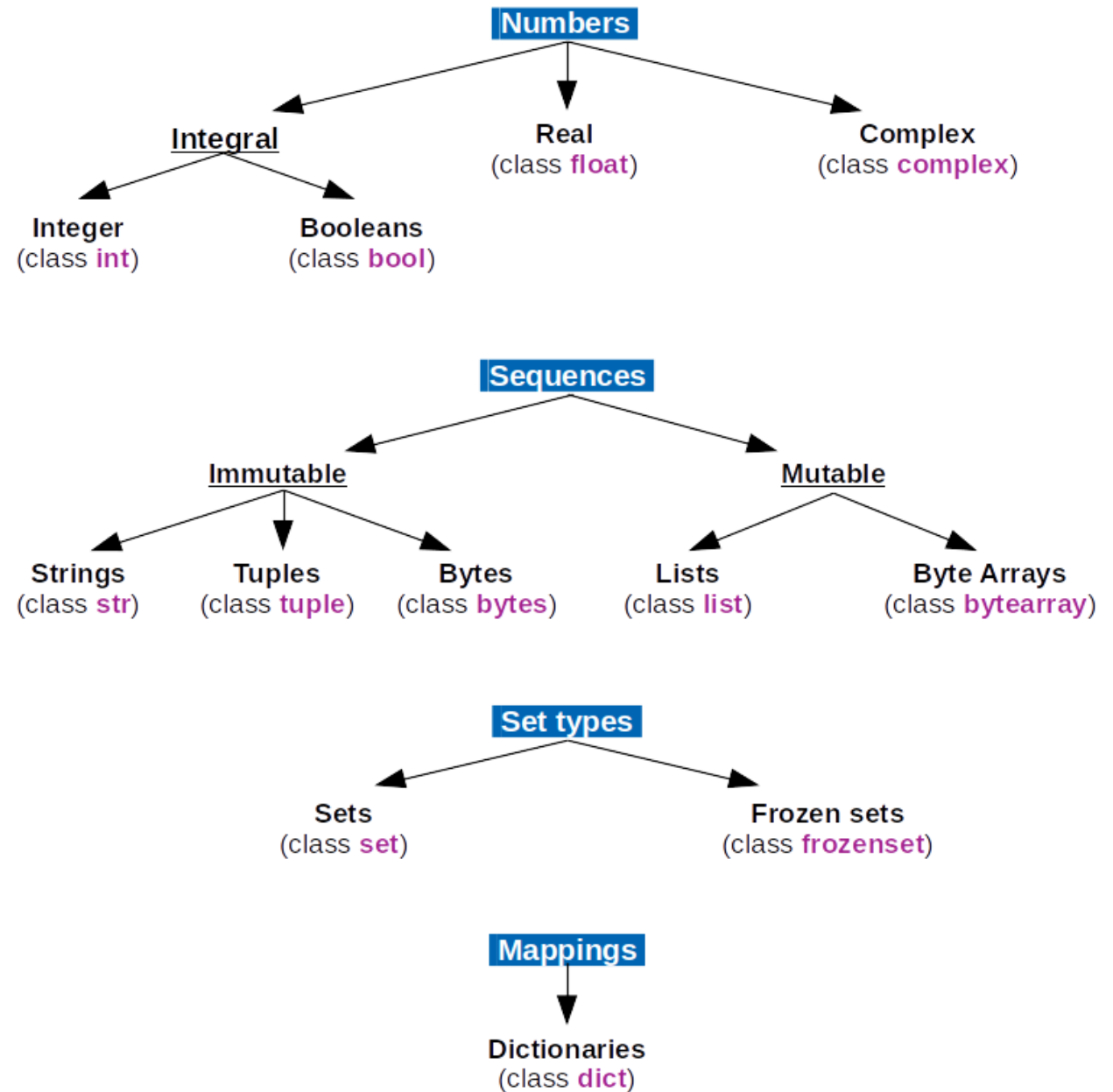


<https://github.com/ARTS-Laboratory/dataset-free-vibration-of-cantilever-beam>

# Python

Key types in Python are:

- Numbers:
  - Integer
  - Booleans
  - Real
- Sequences:
  - Strings " or ""
  - Lists []
  - Tuples ()
- Mappings
  - Dictionaries { }



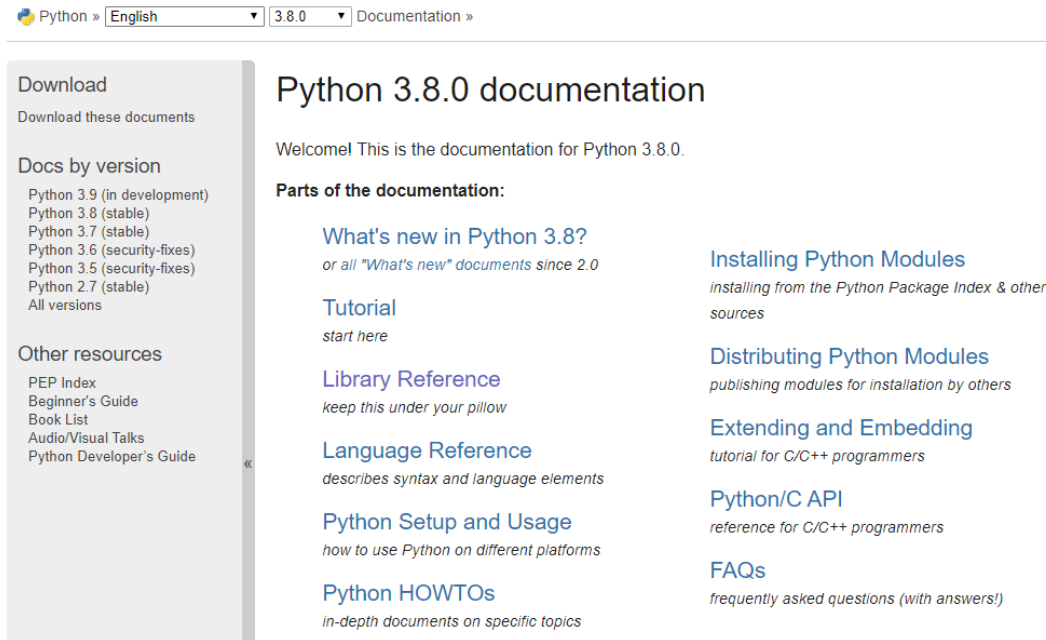
## Example #2 – Pure Python



# Additional Python Resources

Python documentation

<https://docs.python.org/3.8/>



The screenshot shows the Python 3.8.0 documentation page. At the top, there's a navigation bar with 'Python »', a language dropdown set to 'English', a version dropdown set to '3.8.0', and a 'Documentation »' link. Below this, the page title is 'Python 3.8.0 documentation'. A welcome message says 'Welcome! This is the documentation for Python 3.8.0.' Under 'Parts of the documentation:', there are several links: 'What's new in Python 3.8?' (with a subtitle 'or all "What's new" documents since 2.0'), 'Installing Python Modules' (with a subtitle 'installing from the Python Package Index & other sources'), 'Distributing Python Modules' (with a subtitle 'publishing modules for installation by others'), 'Extending and Embedding' (with a subtitle 'tutorial for C/C++ programmers'), 'Python/C API' (with a subtitle 'reference for C/C++ programmers'), 'FAQs' (with a subtitle 'frequently asked questions (with answers!)'), 'Tutorial' (with a subtitle 'start here'), 'Library Reference' (with a subtitle 'keep this under your pillow'), 'Language Reference' (with a subtitle 'describes syntax and language elements'), 'Python Setup and Usage' (with a subtitle 'how to use Python on different platforms'), and 'Python HOWTOs' (with a subtitle 'in-depth documents on specific topics'). On the left side, there's a sidebar with 'Download' (with a subtitle 'Download these documents'), 'Docs by version' (listing Python 3.9 (in development), Python 3.8 (stable), Python 3.7 (stable), Python 3.6 (security-fixes), Python 3.5 (security-fixes), Python 2.7 (stable), and All versions), and 'Other resources' (listing PEP Index, Beginner's Guide, Book List, Audio/Visual Talks, and Python Developer's Guide).

YouTube : “Python Programming” by Derek Banas

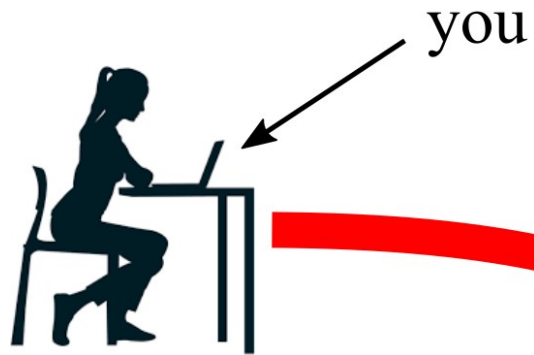
<https://www.youtube.com/watch?v=N4mEzFDjqtA>



And many many more.....



# Our Ministack



**IP[y]:** IPython  
Interactive Computing

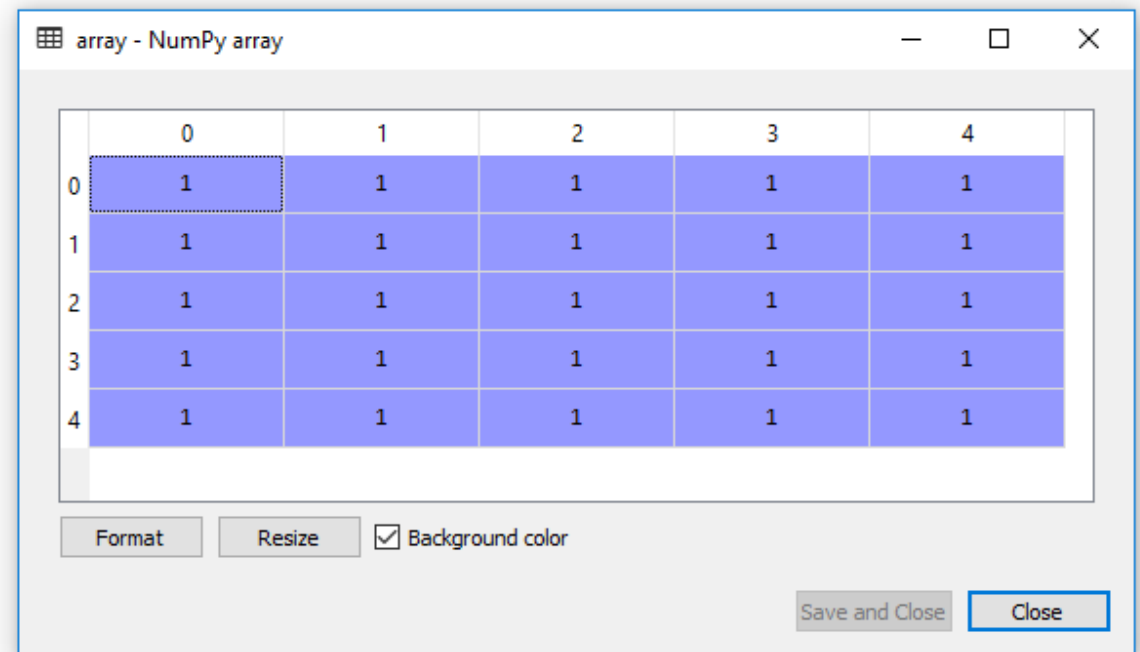


# NumPy (Numerical Python)

- Python does not support arrays or matrices and its mathematical support is limited and slow.
- NumPy was created to add support for arrays and matrices to Python.
- Numpy adds support for high-level mathematical functions to operate on these arrays and matrices.
- Python documentation can be found at <https://numpy.org/doc/1.17/reference/index.html>
- Numpy is limited in scope and is only designed to add numerical computing to python.
- NumPy underpins nearly every scientific computing platform created for Python.



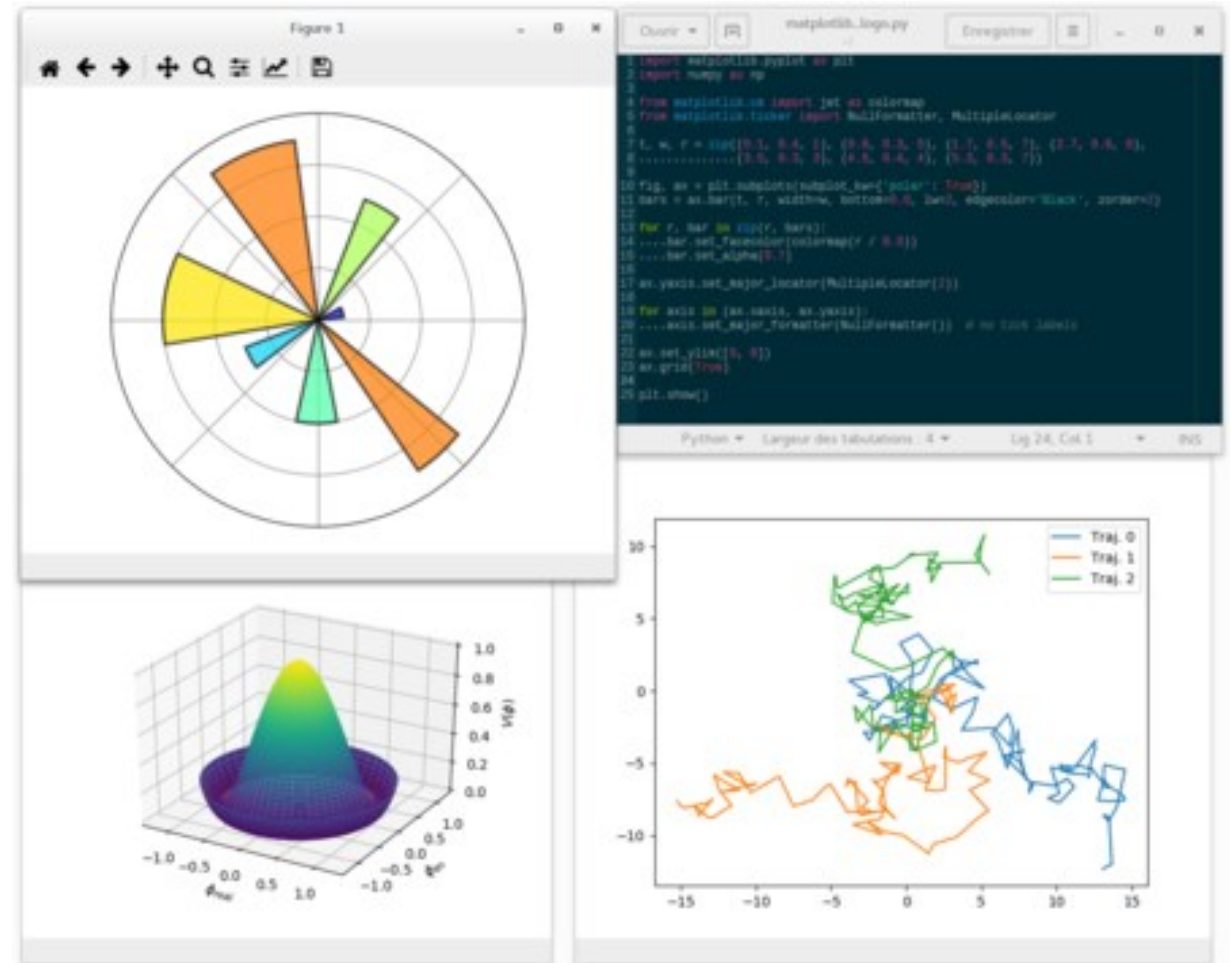
```
39 # import numpy as np
40 import numpy as np
41
42 # build an array
43 array = np.ones((5,5))
```

A screenshot of a window titled "array - NumPy array". It displays a 5x5 grid of blue cells, each containing the number "1". The grid has column headers 0-4 and row headers 0-4. The cell at row 0, column 0 is highlighted with a dashed border. Below the grid are buttons for "Format", "Resize", and a checked "Background color" checkbox. At the bottom right are "Save and Close" and "Close" buttons.

|   | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| 0 | 1 | 1 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 | 1 | 1 |
| 2 | 1 | 1 | 1 | 1 | 1 |
| 3 | 1 | 1 | 1 | 1 | 1 |
| 4 | 1 | 1 | 1 | 1 | 1 |

# Matplotlib

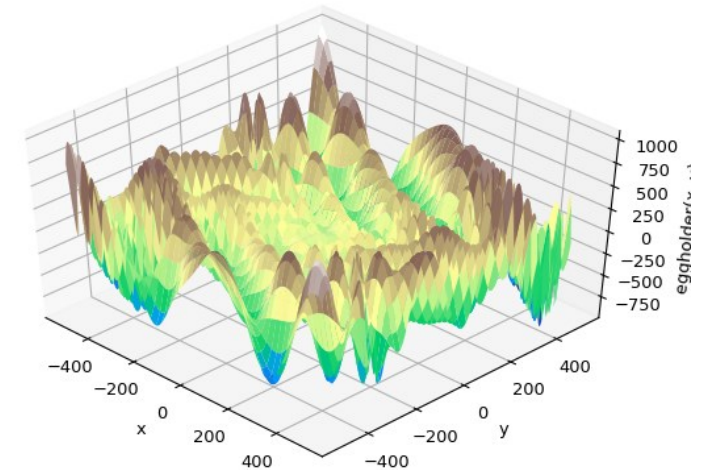
- Matplotlib is a plotting library for Python.
- At one point it was formed by combining multiple libraries (pyplot and pylab). We will use pyplot as pylabs use is discouraged.
- Pyplot is a Matplotlib module which provides a MATLAB-like interface.
- matplotlib documentation can be found at <https://matplotlib.org/3.1.1/api/index.html>
- Pyplot documentation can be found at [https://matplotlib.org/3.1.1/api/\\_as\\_gen/matplotlib.pyplot.html#module-matplotlib.pyplot](https://matplotlib.org/3.1.1/api/_as_gen/matplotlib.pyplot.html#module-matplotlib.pyplot)



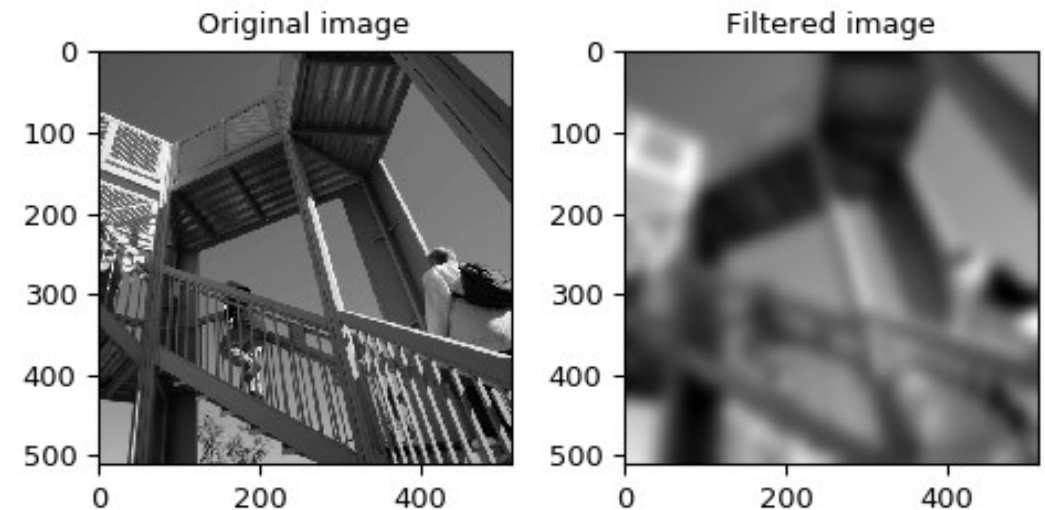
# SciPy (not ScientificPython but close enough)

- SciPy is a Python library used for scientific computing and technical computing.
- SciPy builds on the NumPy array class and the capabilities of matplotlib.
- SciPy contains modules for optimization, linear algebra, integration, interpolation, special functions, FFT, signal and image processing, ODE solvers and other tasks common in science and engineering.
- SciPy and NumPy contain some duplicate capabilities (e.g., FFT). A common way to look at this is that the NumPy version is more bare-bones than the equivalent SciPy version.
- SciPy documentation can be found at <https://docs.scipy.org/doc/scipy/reference/>

## Optimization (scipy.optimize)



## Signal Processing (scipy.signal)



# scikit-learn (also called sklearn)

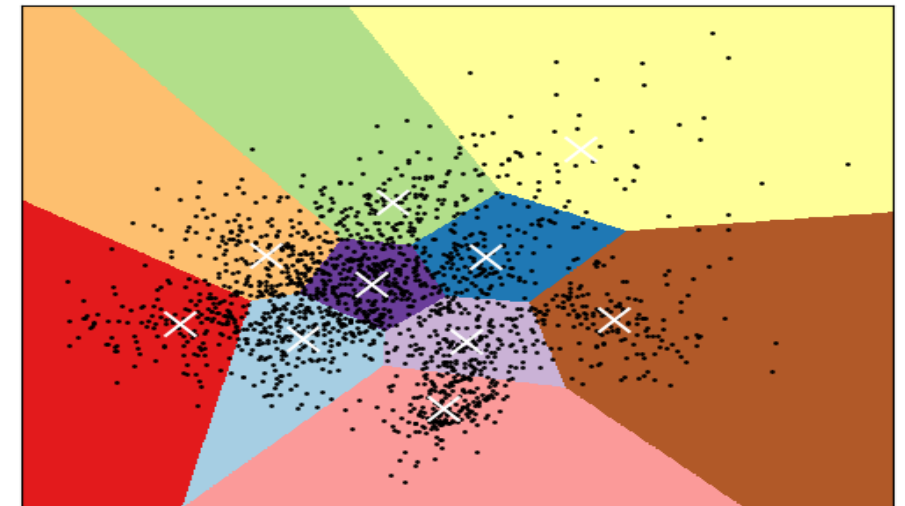
- Sklearn is a machine learning library for Python.
- It features various classification, regression and clustering algorithms including support vector machines, random forests, gradient boosting, k-means and DBSCAN.
- Sklearn is designed to interoperate with the Python numerical and scientific libraries NumPy and SciPy.
- Sklearn documentation can be found at [https://scikit-learn.org/stable/user\\_guide.html](https://scikit-learn.org/stable/user_guide.html)



## Decision Trees



## Clustering



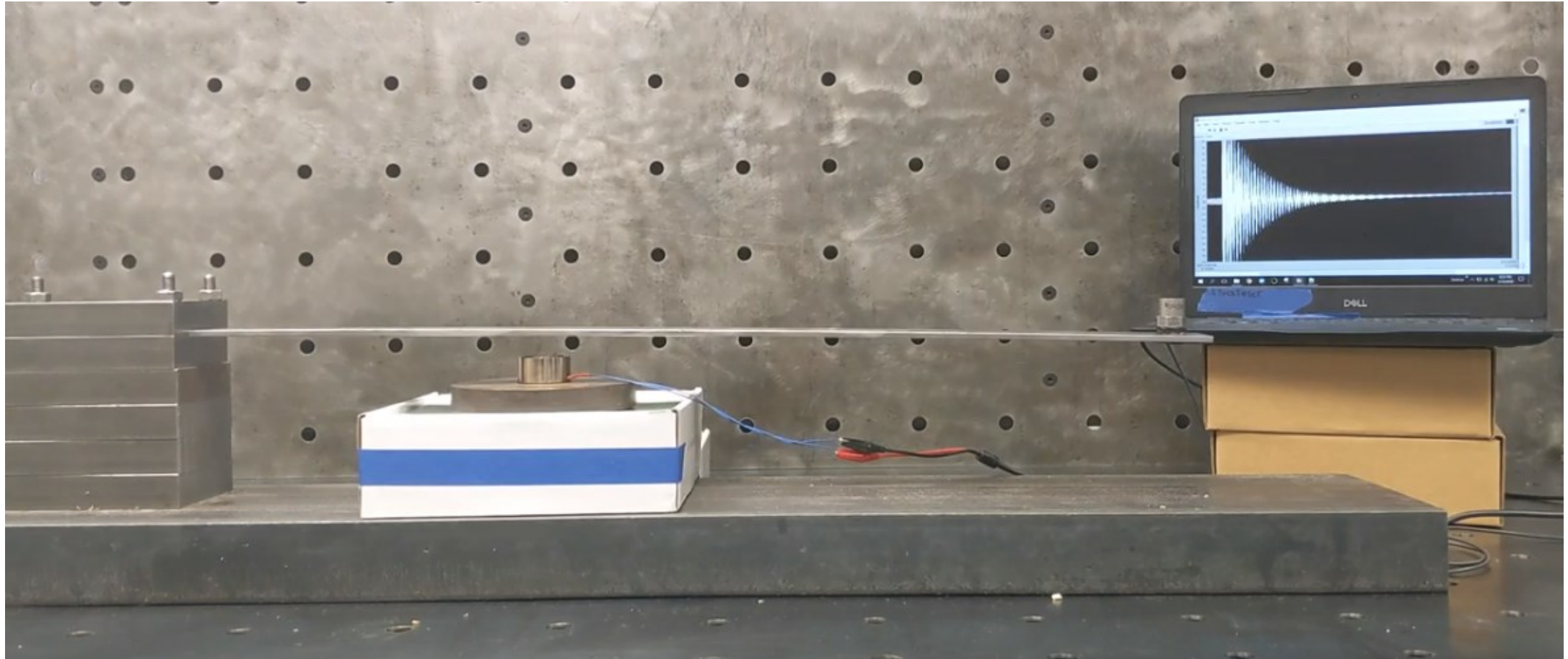
# Conclusions:

- Python is great, but not useful for scientific computing.
- Scientific modules have been developed for python to expand its usefulness into scientific computing. The major packages are:
  - NumPy
  - Matplotlib
    - Pyplot
  - SciPy
  - Sklearn
- We code in a Integrated Development Environment (IDE) of which there are lots, we used Spyder in our examples.

```
1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-
3 """
4 Created on Fri Nov 15 16:05:28 2019
5
6 @author: austin
7 """
8
9
10 import IPython as IP
11 IP.get_ipython().magic('reset -sf')
12
13 import numpy as np
14 import scipy as sp
15 import matplotlib as mpl
16 import matplotlib.pyplot as plt
17
18 plt.close('all')
19
20 ### Load and plot data
21 D = np.loadtxt('vibration_data/Vibration_measurement.csv',delimiter=',')
22
23 tt = D[:,0]
24 dd = D[:,1]
25
26 plt.figure('beam data',figsize=(3.5,3))
27 plt.plot(tt,dd,'-',label='data 1')
28 plt.grid(True)
29 plt.xlabel('time (s)')
30 plt.ylabel('voltage (v)')
31 plt.title('beam data')
32 plt.xlim([-0.1,1.6])
33 plt.legend(framealpha=1,loc=8)
34 plt.tight_layout()
35 plt.savefig('plot.png',dpi=300)
36 array = np.ones((5,5))
37
38 ### Plot an FFT of the data
39
```



## Example #3 – Data processing (FFT) in SciPy



# Thank You for Your Time

Name: Austin R.J. Downey  
Title: Associate Professor  
Email: [austindowney@sc.edu](mailto:austindowney@sc.edu)  
Lab GitHub: [github.com/arts-laboratory](https://github.com/arts-laboratory)



**Molinaroli College of  
Engineering and Computing**  
UNIVERSITY OF SOUTH CAROLINA